Benchmarking Recursive Language Models Against Plain LLMs Under Local Ollama Inference

Abstract

Recursive Language Models (RLMs) aim to improve problem solving by allowing a model to iteratively inspect context, execute code, and call subordinate reasoning processes instead of producing a single direct completion. This paper reports a local benchmark comparing plain large language model (LLM) baselines against RLM execution on a shared set of seven tasks using an on-device Ollama deployment of `qwen2.5:7b`. The benchmark includes short-context question answering, long-context retrieval, document reasoning, planning, and two traveling salesman problem (TSP) style reasoning tasks. Each task was run five times in matched `LLM` and `RLM` conditions, and full terminal outputs were preserved as markdown artifacts. Across the full local benchmark, `RLM` was consistently more expensive than the paired `LLM` baseline in wall-clock latency and token usage, with especially large overheads on the TSP reasoning tasks. The largest average local gap appeared on stochastic adaptive TSP, where `RLM` averaged `843.710` seconds and `27,690.5` tokens over successful runs, compared with `164.309` seconds and `1,370.6` tokens for `LLM`. One `RLM` run in that task failed with exit code `-9`. These results should be interpreted as a local systems benchmark of `LLM` versus `RLM` behavior under a single open-weight model rather than as a substitute for hosted-model comparisons.

1. Introduction

Large language models are commonly evaluated in a single-pass setting: a prompt is presented, the model produces a completion, and the interaction ends. Recursive Language Models change this interaction pattern. Instead of treating the model as a one-shot generator, an RLM is allowed to reason over multiple iterations, inspect execution state, run code, and potentially spawn child reasoning calls. In principle, this can improve groundedness, decomposition, and multi-step reasoning. In practice, it also introduces computational overhead and new failure modes.

This paper focuses on a concrete systems question: what happens when the same open-weight model is used in both a plain `LLM` configuration and an `RLM` configuration on a local machine? To answer that question, we ran a paired benchmark suite using Ollama with `qwen2.5:7b` as the underlying model. The tasks were selected to span:

1. short direct question answering
2. long-context retrieval with distractors
3. long-context clinical extraction
4. app-planning and structured reasoning

5. document-based warning extraction
6. hallucination resistance under missing information
7. explicit stochastic reasoning

The contribution of this paper is narrow and practical:

- it presents a reproducible local benchmark configuration
- it reports raw latency and token usage for both `LLM` and `RLM`
- it preserves per-run outputs for auditability
- it isolates the findings to a local Ollama run and excludes prior Gemini-based runs

2. Background

2.1 Plain LLM Execution

In the baseline setting, a single prompt is sent directly to the model. The model returns one answer, and the benchmark records wall time and token usage. This is the simplest and cheapest evaluation path in the repository.

2.2 Recursive Language Models

In the `RLM` setting used here, the model interacts through a recursive execution loop. The root model can:

- inspect the provided `context`
- write and execute Python in a local REPL
- call `FINAL_VAR(...)` to return a final answer
- in some configurations, call child `RLM` or `LLM` functions

This iterative design enables structured reasoning and self-correction, but it also increases latency and context accumulation. The local benchmark is therefore useful not just for correctness but also for profiling the cost of recursion.

3. Experimental Setup

3.1 Hardware and Software Environment

The benchmark was run locally in the following repository:

- Workspace: `/Users/abhigyanshekhar/Desktop/RLM-FULL`

The local inference stack was:

- Backend: `ollama`
- Endpoint: `http://127.0.0.1:11434/v1`
- Model: `qwen2.5:7b`
- Python environment: repository `.venv`

The benchmark runner used matched paired execution:

```
.venv/bin/python run_benchmark_pairs.py --backend ollama --model qwen2.5:7b --runs 5
```

3.2 Benchmarks

Seven benchmark pairs were included:

1. **Battle of the Bastards** A short context question asking for the main character and decisive ally.
2. **AuthProxy Long Context** Long-context retrieval and distractor resistance over service configuration data.
3. **Clinical Long Context** Long-context extraction over a clinical-style benchmark prompt.
4. **Launch Note App** Structured planning and implementation-style reasoning.
5. **PDF Cersei Warning** Document reasoning over a PDF-derived benchmark context.
6. **Under-specified TSP** A hallucination-resistance task in which the distance matrix is intentionally missing.
7. **Stochastic Adaptive TSP** A fully specified stochastic optimization problem requiring exact adaptive reasoning.

3.3 Logging and Artifact Preservation

Every individual benchmark execution was saved as a markdown file. The final local batch folder is:

- `/Users/abhigyanshekhar/Desktop/RLM-FULL/benchmark_runs/20260403-121151-ollama-qwen2.5-7`

This folder contains:

- one subfolder per benchmark
- one `.md` file per `LLM` run
- one `.md` file per `RLM` run
- a batch `SUMMARY.md`
- a machine-readable `raw_data.csv`
- a machine-readable `aggregate_metrics.csv`

This design makes the benchmark auditable at the per-run level.

3.4 Measurement

For each run, we extracted:

- exit code
- wall-clock time
- input tokens
- output tokens
- total tokens

For `RLM`, we also preserved the internal execution trace and the reported `RLM execution time` when available. One `RLM` sample, `stochastic_tsp` run `004`,

exited with code `-9`. That run was preserved in the raw data and excluded from the aggregate averages for successful `RLM` stochastic TSP runs.

4. Results

4.1 Aggregate Metrics

Benchmark	Mode	Success	Avg Wall (s)	Median Wall (s)	Avg Input Tokens	Avg Output Tokens	Avg Total Tokens
Battle of the Bastards	LLM	5/5 (100%)	7.803	7.756	275.0	38.6	313.6
Battle of the Bastards	RLM	5/5 (100%)	108.379	73.517	9145.0	440.0	9585.0
AuthProxy Long Context	LLM	5/5 (100%)	23.689	23.109	1082.0	166.0	1248.0
AuthProxy Long Context	RLM	5/5 (100%)	179.107	173.042	11749.0	1251.6	13000.6
Clinical Long Context	LLM	5/5 (100%)	104.282	111.993	1645.0	1179.6	2824.6
Clinical Long Context	RLM	5/5 (100%)	137.302	52.487	11503.8	1017.2	12521.0
Launch Note App	LLM	5/5 (100%)	91.281	81.166	257.0	1044.6	1301.6
Launch Note App	RLM	5/5 (100%)	273.112	276.742	10956.8	2154.2	13111.0
PDF Cersei Warning	LLM	5/5 (100%)	132.849	133.707	4096.0	912.4	5008.4

Benchmark	Mode	Success	Avg Wall (s)	Median Wall (s)	Avg Input Tokens	Avg Output Tokens	Avg Total Tokens
PDF Cersei Warning	RLM	5/5 (100%)	212.396	191.413	19444.4	951.6	20396.0
Under-specified TSP	LLM	5/5 (100%)	108.994	106.786	209.0	1115.6	1324.6
Under-specified TSP	RLM	5/5 (100%)	510.680	369.738	20109.2	3406.0	23515.2
Stochastic Adaptive TSP	LLM	5/5 (100%)	164.309	185.630	318.0	1052.6	1370.6
Stochastic Adaptive TSP	RLM	4/5 (80%)	843.710	700.832	24390.0	3300.5	27690.5

4.2 Wall-Time Comparison

The shortest average baseline condition was Battle of the Bastards `LLM` at `7.803` seconds. The longest average successful recursive condition was Stochastic Adaptive TSP `RLM` at `843.710` seconds. Even on relatively simple tasks, recursive execution introduced substantial local delay. For example:

- Battle of the Bastards: `7.803s` (`LLM`) vs `108.379s` (`RLM`)
- AuthProxy: `23.689s` (`LLM`) vs `179.107s` (`RLM`)
- Under-specified TSP: `108.994s` (`LLM`) vs `510.680s` (`RLM`)

4.3 Token Usage Comparison

The token overhead of recursion was also large throughout the benchmark:

- Battle of the Bastards: `313.6` total tokens (`LLM`) vs `9585.0` (`RLM`)
- AuthProxy: `1248.0` vs `13000.6`
- PDF Cersei Warning: `5008.4` vs `20396.0`
- Under-specified TSP: `1324.6` vs `23515.2`
- Stochastic Adaptive TSP: `1370.6` vs `27690.5` over successful `RLM` runs

The local data therefore show a consistent pattern: `RLM` consumed substantially more tokens than the matched `LLM` baseline in every task.

4.4 Reliability Note

All `LLM` runs completed successfully in this batch. All `RLM` runs also completed successfully except one:

- `stochastic_tsp` run `004`
- exit code: `-9`

This failed sample is important because it occurred on the hardest recursive reasoning task in the suite.

5. Discussion

5.1 Systems Cost of Recursion

The most immediate finding is that local recursion is expensive. Even when `RLM` produces a usable answer, the cost in latency and tokens is far above the corresponding baseline. On a local Ollama deployment with `qwen2.5:7b`, this means that recursion should be treated as a limited budgeted resource rather than the default interaction mode.

5.2 Task Type Matters

The local overhead varied by task type:

- On short, direct tasks, the absolute cost remained moderate, but the relative overhead was still large.
- On long-context retrieval tasks, the recursive path amplified both context usage and runtime.
- On explicit reasoning problems such as the TSP tasks, the recursive cost became extreme.

This suggests that the local usefulness of `RLM` is likely task-dependent. If a task can already be answered reliably by a direct pass, recursion may not be justified under local compute constraints.

5.3 Interpretation Limits

This benchmark paper intentionally emphasizes measurable systems characteristics. It does not claim that `RLM` is always worse, nor that `LLM` is always sufficient. It shows that under this particular local model and hardware setting, recursive execution carries a large runtime and token overhead. The more interesting question for future work is whether that overhead buys better answer quality often enough to justify itself.

6. Threats to Validity

Several limitations constrain the interpretation of these results.

6.1 Single Model Family

All local results here use only `qwen2.5:7b`. A larger or faster local model might change the magnitude of the observed overheads.

6.2 Local Deployment Specificity

These are local Ollama results, not hosted inference results. They should not be mixed with earlier Gemini-based findings or treated as a direct replacement for them.

6.3 Limited Repetition

Each task was run five times per mode. This is enough to expose variation and preserve multiple raw traces, but it is still a small sample for strong statistical claims.

6.4 Partial Evaluation Scope

This paper reports latency and token usage directly because those metrics were consistently logged. A full accuracy study would still require a formal scoring rubric, independent judging, and inter-rater agreement.

7. Conclusion

This paper presented a full local benchmark comparing plain `LLM` execution with recursive `RLM` execution under Ollama using `qwen2.5:7b`. Across seven benchmark pairs and five matched runs per pair, `RLM` was consistently slower and more token-intensive than the paired `LLM` baseline. The gap was especially large on the TSP reasoning benchmarks, and one `RLM` run failed outright on the stochastic adaptive TSP task. The main conclusion is therefore operational: under this local open-weight setup, recursive execution imposes a substantial systems cost. Whether that cost is justified depends on answer quality, which should be evaluated in a follow-up scoring study over the preserved raw run logs.

References

1. Local benchmark report: `/Users/abhigyanshekhar/Desktop/RLM-FULL/RLM_Setup_And_Test_Guide.m`
2. Raw local run artifacts: `/Users/abhigyanshekhar/Desktop/RLM-FULL/benchmark_runs/20260403-121`
3. Aggregate CSV: `/Users/abhigyanshekhar/Desktop/RLM-FULL/benchmark_runs/20260403-121151-ol`
4. Raw CSV: `/Users/abhigyanshekhar/Desktop/RLM-FULL/benchmark_runs/20260403-121151-ollama-`